

Reinforcement Learning Reviewer

Modules 9, 10 & 11

Made by James Adrian B. Castro

A comprehensive reviewer covering:

- Dynamic Programming & Planning
- Monte Carlo Tree Search (MCTS)
- Q-Learning & Temporal Difference

MODULE 9

Dynamic Programming, Planning & Reinforcement

1. Markov Decision Processes (MDPs)

MDPs formally describe an environment for reinforcement learning where the environment is fully observable.

Almost all RL problems can be formalized as MDPs:

- Optimal control → continuous MDPs
- Partially observable problems → can be converted into MDPs
- Bandits → MDPs with one state

MDPs are considered 'solved' if an optimal policy is obtained.

The Markov Property

"The future is independent of the past given the present."

$$P[S_{t+1} | S_t] = P[S_{t+1} | S_1, \dots, S_t]$$

- Once the state is known, history may be thrown away
- The current state is a sufficient statistic of the future

MDP Definition

A Markov Decision Process is a tuple $\langle S, A, P, R, \gamma \rangle$:

Symbol	Meaning
S	Finite set of states
A	Finite set of actions
P	State transition probability matrix
R	Reward function
γ	Discount factor (0 to 1)

$$P_{ss'}^a = P[S_{t+1}=s' | S_t=s, A_t=a]$$

$$R_s^a = E[R_{t+1} | S_t=s, A_t=a]$$

- Each row of the state transition matrix sums to 1

Student Value-Function MDP (Example)

Given $\pi(a|s) = 0.5$, $\gamma = 1$:

State	Value
Facebook	-2.3
Quit	-1.3
Study (middle)	2.7
Study (right)	7.4

Sleep

0

2. Dynamic Programming (DP)

DP is a method for solving complex problems by breaking them down into subproblems.

- Solve the subproblems
- Combine solutions to subproblems

DP = collection of algorithms that compute optimal policies given a perfect model of the environment as an MDP.

Fibonacci Example

Formula: $F(n) = F(n-1) + F(n-2)$

Without DP: repeatedly calculates the same values (exponential time).

With DP (memoization):

```
function fibonacci(n, memo = {}) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  memo[n] = fibonacci(n - 1, memo) + fibonacci(n - 2, memo);
  return memo[n];
}
console.log(fibonacci(6)); // Output: 8
```

Two Properties of DP

- Optimal substructure — optimal solution can be decomposed into subproblems
- Overlapping subproblems — subproblems recur many times; solutions can be cached and reused

MDPs satisfy both properties: Bellman equation gives recursive decomposition, and the value function stores and reuses solutions.

DP in MDP: Robot Example

States: $A \rightarrow B \rightarrow C$ (Goal). Reward at $C = +10$, others = 0.

$V(C) = 10$, $V(B) = 0 + 10 = 10$, $V(A) = 0 + 10 = 10$

- Optimal value of A is built from optimal value of B
- Optimal value of B is built from optimal value of C
- Demonstrates Optimal Substructure

Overlapping Subproblems: If A can go to B or C, and both B and C eventually go to D, then $V(D)$ is needed twice. DP computes $V(D)$ once and stores it in a value table for reuse.

3. The Bellman Equation

$V(s) = \max_a [R(s,a) + \gamma V(s')]$

- Breaks a large problem into smaller state-value problems (Optimal Substructure)
- Stores computed state values for reuse (Overlapping Subproblems)

4. Planning

Planning is any computational process that uses a model to create or improve a policy.

Model → Simulated Experience → Backups → Values → Policy

- All state-space planning methods involve computing value functions
- They all apply backups to simulated experience

Direct vs Indirect RL

Type	Purpose
Direct RL	Directly improve value function and policy
Indirect RL	Improve the learned model (model-based RL / planning)

- Indirect (model-based): fuller use of experience; better policy with fewer interactions
- Direct: simpler; not affected by bad models

Planning, acting, model learning, and direct RL can occur simultaneously and in parallel.

5. Planning by DP: Prediction & Control

Component	Description
Prediction	Learn value function for the policy followed
Control	Find a policy that maximizes reward

Weather Example (Farmer)

States: Sunny, Cloudy, Rainy. Actions: Water crops, Do not water crops.

Rewards: Healthy = +10, Wasting water = -5, Damaged = -10

Prediction (Value Function):

Weather	Value
Sunny	+8
Cloudy	+5
Rainy	+2

Control (Optimal Policy):

- Sunny → Water crops
- Cloudy → Check soil first
- Rainy → Do not water

6. Iterative Policy Evaluation

Problem: evaluate a given policy. Solution: iterative application of Bellman expectation formula.

$V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_\pi$

Algorithm:

Input: policy π to be evaluated

Parameter: small threshold $\theta > 0$

Initialize $V(s)$ arbitrarily, $V(\text{terminal}) = 0$

Loop:

$\Delta \leftarrow 0$

```

Loop for each  $s \in S$ :
   $v \leftarrow V(s)$ 
   $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$ 
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
until  $\Delta < \theta$ 

```

7. Policy Improvement & Policy Iteration

Given policy π : evaluate the policy, then improve by acting greedily.

$\pi_0 \rightarrow v_{\{\pi_0\}} \rightarrow \pi_1 \rightarrow v_{\{\pi_1\}} \rightarrow \pi_2 \rightarrow \dots \rightarrow \pi^* \rightarrow v^*$

Each policy is guaranteed to be a strict improvement over the previous one (unless already optimal).

Because a finite MDP has only a finite number of policies, this process must converge to an optimal policy and optimal value function in a finite number of iterations.

Policy Iteration Algorithm:

```

1. Initialize  $V(s)$  and  $\pi(s)$  arbitrarily for all  $s$ 

2. Policy Evaluation
  Loop:
     $\Delta \leftarrow 0$ 
    Loop for each  $s$ :
       $v \leftarrow V(s)$ 
       $V(s) \leftarrow \sum p(s',r|s,\pi(s))[r + \gamma V(s')]$ 
       $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
  until  $\Delta < \theta$ 

3. Policy Improvement
  policy-stable  $\leftarrow$  true
  For each  $s$ :
    old-action  $\leftarrow \pi(s)$ 
     $\pi(s) \leftarrow \operatorname{argmax}_a \sum p(s',r|s,a)[r + \gamma V(s')]$ 
    if old-action  $\neq \pi(s)$ : policy-stable  $\leftarrow$  false
  If policy-stable: stop and return  $V \approx v^*, \pi \approx \pi^*$ 
  Else: go to 2

```

MODULE 10

Monte Carlo Tree Search (MCTS)

1. Introduction

MCTS is a best-first search method that does not require a positional evaluation function. It is based on a randomized exploration of the search space.

- Uses results of previous explorations to gradually build a game tree in memory
- Successively becomes better at estimating values of the most promising moves
- Heavily relies on randomness ('Monte Carlo' from the gambling district in Monaco)

Planning time is independent of size of state space $|S|$.

2. Tic-Tac-Toe Example

Possible moves: A, B, C

Step 1: Randomized Exploration

Move	Wins	Losses	Draws	Win Rate
A	40	35	25	40%
B	60	20	20	60%
C	90	5	5	90%

Step 2: Build a Game Tree

Current State

```

├─ Move A (40%)
├─ Move B (60%)
└─ Move C (90%)
  
```

The tree grows larger as more simulations are performed.

Step 3: Choose the Best Move

Move C has the highest estimated win rate (90%) → MCTS selects Move C.

Chess Example

Move	Simulations	Wins	Win Rate
Pawn	1000	450	45%
Knight	1000	620	62%
Castle	1000	710	71%

3. The Four Phases of MCTS

Four phases iterated for a determined number of times: Selection → Expansion → Simulation →

Backpropagation.

The higher the number of iterations, the more the tree grows and the easier it is for results to converge.

Phase 1: Selection

- Starting at root, select a node based on the biggest UCB value
- UCB formula balances exploitation and exploration:

$$UCB = w_i / s_i + c * \sqrt{\ln(s_p) / s_i}$$

- Exploitation: grow the tree in depth from a promising node
- Exploration: grow the tree in width (check other nodes)

Phase 2: Expansion

- If a node has been visited, its children are generated and added to the tree
- Otherwise, continue to the next phase

```
function EXPANSION(node)
  if is node visited then
    generate children of node
  end if
end function
```

Phase 3: Simulation

- If selected node is expanded, take a random child; else take the selected node itself
- Randomly choose next states until reaching a terminal state

```
function SIMULATION(node)
  while node state is not final do
    get next node random
    node = next node
  end while
  return node state
end function
```

Phase 4: Backpropagation

- The value from simulation is propagated from leaves to root
- Updates the values of the nodes visited

```
function BACKPROPAGATION(node)
  while node is not null do
    increment node score
    increment node visits
    node = parent node
  end while
end function
```

4. Why Randomness Works

Law of Large Numbers (LLN): From a single random game, very little can be learned. But from simulating a multitude of random games, a good strategy can be inferred.

- If White wins 80% of thousands of random games from a state, there must be something about the state giving White an advantage

- Same logic as random initializers in neural networks

5. MCTS Strategy Summary

Aggressive

- Play a move that causes an immediate win (if possible)
- Play a move that sets up a future winning situation

Defensive

- Play a move that prevents opponent from winning next round (if possible)
- Play a move that prevents opponent from setting up a future winning situation

6. Advantages of MCTS

- Universally applicable — no evaluation function necessary
- Useful even for games with large state spaces (e.g., Go)
- Planning time is independent of size of states $|S|$

MODULE 11

Q-Learning & Temporal Difference

1. Monte Carlo Limitation

Monte Carlo RL overcomes unknown model difficulty, but a disadvantage is that the strategy can only be updated after the whole episode.

- MC must wait until the end of the episode before the return is known
- High variance, low bias
- Does not exploit the Markov property

2. Temporal Difference (TD) Learning

TD Learning is a combination of Monte Carlo ideas and Dynamic Programming ideas.

- Like MC: can learn directly from raw experience without a model
- Like DP: updates estimates based in part on other learned estimates, without waiting for a final outcome

In simple terms: TD Learning learns by comparing what the agent expected to happen with what actually happened.

TD Update Formula

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

TD Error: $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$

Symbol	Meaning
$V(S_t)$	Current estimated value
R_{t+1}	Reward received
$V(S_{t+1})$	Estimated value of next state
α	Learning rate
γ	Discount factor

TD(0) Algorithm

Input: policy π to be evaluated

Parameter: step size $\alpha \in (0,1]$

Initialize $V(s)$ arbitrarily, $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

 until S is terminal

MC vs TD Comparison

Monte Carlo	Temporal Difference
Must wait until end of episode	Can learn online after every step
High variance, low bias	Low variance, some bias
Does not exploit Markov property	Exploits Markov property
No bootstrapping	Bootstraps (guess toward a guess)
Model-free	Model-free

Key TD Properties

- Learns directly from episodes of experience (online/incremental)
- Model-free — no knowledge of MDP transitions/rewards
- Learns from incomplete episodes by bootstrapping
- Updates a guess towards a guess
- Exploits the Markov property
- Converges faster than constant MC in practice

Driving Home Example

State	Elapsed	Predicted to Go	Predicted Total
Leaving office	0	30	30
Reach car, raining	5	35	40
Exiting highway	20	15	35
2ndary road, truck	30	10	40
Entering home street	40	3	43
Arrive home	43	0	43

- MC methods: update only at the end (actual outcome = 43)
- TD methods: update after each step (bootstrapping)

3. Behavior Policy vs Target Policy

Policy	Definition
Behavior Policy	Policy the agent uses for action selection (interacts with env)
Target Policy	Policy the agent is trying to learn (value function for this policy)

4. On-Policy vs Off-Policy Learning

On-Policy	Off-Policy
'Learn on the job'	'Look over someone's shoulder'
Target policy == Behavior policy	Target policy != Behavior policy
Experiences collected using latest policy	Can use older samples (from older policies)
Agent interacts with environment	Learn from experience sampled from another
Online	Can learn from demonstration

5. Q-Learning

Q stands for Quality — how useful a given action is in gaining future reward.

- Learned action-value function Q directly approximates the optimal Q^*
- Seeks to find the best action given the current state
- Dramatically simplifies analysis and enables early convergence
- Does not require a model of the environment (model-free)

Q-Learning Update

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Term	Meaning
$Q(s,a)$	Current Q-value
α	Learning rate
r	Reward
γ	Discount rate
$\max_{a'} Q(s',a')$	Maximum expected future reward

Q-Learning Algorithm

```

Initialize  $Q(s,a)$  arbitrarily
Repeat (for each episode):
  Initialize  $s$ 
  Repeat (for each step of episode):
    Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
    Take action  $a$ , observe  $r, s'$ 
     $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ 
     $s \leftarrow s'$ 
  until  $s$  is terminal
  
```

Q-Learning Flowchart

```

Initialize Q-table
↓
Choose an Action
↓
Perform an Action
↓
Measure Reward
↓
Update Q-table → (loop back to Choose an Action)
  
```

A number of iterations result in a good Q-table.

Q-Table

- A table of states $\times$ actions initialized to zero
- Each cell is updated through training
- Example dimensions: 500 states $\times$ 6 actions

6. Deep Q-Learning (DQN)

When the state space is too large for a Q-table, use a neural network to approximate Q-values.

Q-Learning	Deep Q-Learning
Q-table	Neural network
State + Action $\rightarrow$ Q-value	State $\rightarrow$ Q-values for all actions
Tabular	Function approximation

DQN Architecture

State $\rightarrow$ [Neural Network] $\rightarrow$ Q-value Action 1
 $\rightarrow$ Q-value Action 2
 $\rightarrow$...
 $\rightarrow$ Q-value Action N

DQN Components

- Q eval network — computes $Q(s,a; \theta)$
- Target network — computes $\max_{a'} Q(s',a'; \theta')$
- Replay Memory — stores (s, a, r, s') transitions
- DQN loss function — gradient w.r.t. loss

Training Loop

- Environment produces (s, a, r, s')
- Store in replay memory
- Sample batch from replay memory
- Compute $Q(s,a; \theta)$ from eval network
- Compute $\max_{a'} Q(s',a'; \theta')$ from target network
- Update parameters via gradient descent
- Periodically update target network parameters

Key Formulas Summary

Concept	Formula
Markov Property	$P[S_{t+1} S_t] = P[S_{t+1} S_1, \dots, S_t]$
State Transition	$P_{ss'}^a = P[S_{t+1}=s' S_t=s, A_t=a]$
Bellman Equation	$V(s) = \max_a [R(s,a) + \gamma V(s')]$
Policy Evaluation	$V(s) \leftarrow \sum_a \pi(a s) \sum p(s',r s,a)[r + \gamma V(s')]$
Policy Improvement	$\pi'(s) = \operatorname{argmax}_a \sum p(s',r s,a)[r + \gamma V(s')]$
TD Update	$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$
Q-Learning	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
UCB	$w_i/s_i + c * \sqrt{\ln(s_p/s_i)}$